COUNTING ARNOLD'S MEANDERS WITH VERIFIED ALGORITHMS

ALEXY KHRABROV

Abstract. In 1988 V. I. Arnold asked in how many ways a river can cross a straight road n times. No closed formula for these *meander numbers* is known, and the values are known only by computer enumeration. We formalize the problem in the Lean 4 proof assistant and give two counting algorithms that are proved, for every n, to return exactly the number defined by the specification. The first is a depth-first search over crossing orders with a pruning rule based on a parity argument. The second is a transfer matrix that scans the road from west to east, tracks how the open strands of the river connect, merges equal states, and discards states that can no longer be completed. Its correctness proof is a bijection between the action sequences it accepts and meanders, established through a decorated version of the machine that carries the actual pieces of river. Along the way we prove that closed meanders of order n are equinumerous with open meanders with $2n-1$ crossings, and that there are at most C_n^2 closed meanders, where C_n is the n-th Catalan number. The verified transfer matrix computes the meander numbers up to $n = 30$ in about half a minute; values up to $n = 13$ are checked by the Lean kernel alone. Unverified parallel implementations of the same machine in Rust and OxCaml, with each state packed into one machine word, reproduce the published values up to $n = 50$. A meet-in-the-middle algorithm, whose decomposition is proved in Lean, combines forward and backward layers kept on disk and extends the computation to $n = 58$. It gives three new terms of the sequence, each confirmed by two independent decompositions in two separately written programs. Companion notebooks implement the algorithms in Lean, Python and OCaml. The development is about 5,000 lines of Lean and is available at `https://github.com/querygraph/meander`.

1. Introduction

A *meander* is a non-self-intersecting curve in the plane that crosses a fixed straight line a prescribed number of times. In a short note on the topology of real algebraic curves, Arnold [2] posed the enumeration question in its most vivid form: in how many ways can a river cross a straight road n times? Two rivers are considered the same if one can be deformed into the other without changing the order of the crossings along the road and along the river. The resulting numbers,

$$1,\ 1,\ 1,\ 2,\ 3,\ 8,\ 14,\ 42,\ 81,\ 262,\ 538,\ 1828,\ 3926,\ 13820,\ \ldots$$

form sequence A005316 of the OEIS [11]. The closely related count of closed curves crossing the line $2n$ times is A005315 [12]. Meanders appeared earlier in work on folding a strip of stamps [14], were studied systematically by Lando and Zvonkin [8], and became a test bed for exact enumeration and statistical physics [4, 5, 6].

Despite this attention there is no closed formula. Di Francesco, Golinelli and Guitter conjectured from a conformal field theory argument that the number of closed meanders of order n grows like $C\,R^{2n}n^{-\alpha}$ with $R^2 \approx 12.26$ and $\alpha = (29+\sqrt{145})/12$ [5]. Rigorous bounds on the growth rate are far apart: Albert and Paterson proved that the exponential growth constant R^2 lies between about 11.38 and 12.90 [1]. Every known value of the sequence comes from a computer program, and the best of these are intricate transfer-matrix codes [6].

Date: October 2026.

This paper asks a narrower question: can we compute meander numbers with programs whose correctness is machine-checked against the plain definition? We answer it in the Lean 4 proof assistant [9] with the mathlib library [10]. Our contributions are:

(1) A formal specification of open meanders as permutations of the bridges satisfying a non-crossing condition, and of closed meanders (Section 2).
(2) Structural theorems: open meanders exist for every n; there are at most $n!$; closed meanders of order n correspond bijectively to open meanders with $2n-1$ crossings; and there are at most C_n^2 closed meanders (Section 3).
(3) A depth-first search with a pruning rule derived from a parity argument, proved equal to the specification (Section 4).
(4) A transfer matrix with state merging and a capacity-based pruning rule, proved equal to the specification by a bijection between accepted action sequences and meanders (Section 5).
(5) Certified values: kernel-checked for $n \le 13$, and computed by the verified transfer matrix up to $n = 32$, matching the OEIS (Section 6).
(6) Parallel implementations of the same machine in Rust and OxCaml, unverified but checked against the certified values and the OEIS, which reach $n = 50$; teaching implementations in Python and OCaml; and a comparison of all of them (Section 6.1).
(7) A meet-in-the-middle algorithm, with its decomposition proved in Lean, implemented in both languages as an extensible store of layers on disk. It reaches $n = 58$, giving the new terms M_{56}, M_{57} and M_{58} (Section 6.2).

The published sequence ends at $n = 55$ [11]. Our values up to $n = 55$ reproduce it. The three values beyond it are new. Each is computed through two independent splits of the meet-in-the-middle sum, by two separately written programs, all four results agreeing. What is also new is that the smaller values come from programs proved correct for all n, and that the proofs expose the combinatorial facts that make the fast algorithms work.

2. The specification

2.1. **From a picture to a permutation.** Label the bridges $0, 1, \ldots, n-1$ from west to east. Following Arnold, the river enters from the south and leaves to the east. Between consecutive crossings it runs along an arc in one of the two half-planes, alternately: the first arc, from the south, lies below the road, the next above, and so on. A river is therefore determined up to deformation by its *crossing order*, the permutation τ with $\tau(i)$ the bridge crossed i-th.

Not every permutation is realized. To state the condition, compactify each half-plane to a disk whose boundary consists of the road and one point at infinity, and record the two ends of the river as two boundary points east of every bridge: the east end $E = n$ and the south end $S = n + 1$. (Going around the lower disk counterclockwise, one meets the bridges, then east, then south.) The river's path is the sequence of boundary points

$$P_0 = S, \quad P_1 = \tau(0), \ \ldots, \ P_n = \tau(n-1), \quad P_{n+1} = E,$$

and arc k joins P_k and P_{k+1} in the lower half-plane if k is even and the upper one if k is odd. By the Jordan curve theorem the river is simple exactly when no two arcs on the same side cross, and two chords of a disk cross exactly when their endpoints alternate around the boundary. Because all endpoints are now points of a line, alternation is an elementary condition.

Definition 2.1. Chords $\{a, b\}$ and $\{c, d\}$ *interleave* if exactly one of c, d lies strictly between $\min(a, b)$ and $\max(a, b)$. A point sequence P has *no crossing among its first L arcs* if for all $j < k < L$ with $j \equiv k \pmod 2$, the arcs $\{P_j, P_{j+1}\}$ and $\{P_k, P_{k+1}\}$ do not interleave. A

```
def Interleave (a b c d : Nat) : Prop :=
  ¬ ((min a b < c ∧ c < max a b) ↔ (min a b < d ∧ d < max a b))

def NoCross (P : Nat → Nat) (L : Nat) : Prop :=
  ∀ k < L, ∀ j < k, j % 2 = k % 2 →
    ¬ Interleave (P j) (P (j + 1)) (P k) (P (k + 1))

def IsMeander {n : ℕ} (σ : Equiv.Perm (Fin n)) : Prop :=
  NoCross (pathPt σ) (n + 1)

def openMeanderCount (n : ℕ) : ℕ :=
  Fintype.card {σ : Equiv.Perm (Fin n) // IsMeander σ}
```

FIGURE 1. The specification in Lean 4. `pathPt` σ is the point sequence P of the river with crossing order σ.

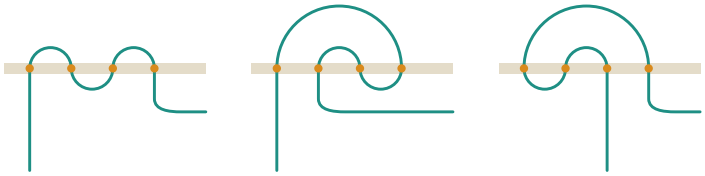


FIGURE 2. The three open meanders with $n = 4$ crossings. The river enters from the south and leaves to the east; bridges are numbered 0 to 3 from west to east. Their crossing orders are 0123, 0321 and 2103.

permutation τ of $\{0, \ldots, n-1\}$ is an *open meander* if its path has no crossing among its $n+1$ arcs. We write M_n for the number of open meanders.

Figure 1 gives the specification in Lean. Figure 2 shows the three open meanders with four crossings, and Figure 3 shows all eight with five.

2.2. **Closed meanders.** A closed meander of order n is a simple closed curve crossing the road $2n$ times. To count each curve once we fix where it starts and in which direction it runs: it starts at the westmost bridge and leaves it into the upper half-plane. It is then a permutation σ of $\{0, \ldots, 2n-1\}$ with $\sigma(0) = 0$ whose cyclic path has no crossing among its $2n$ arcs. We write $\overline{\mathrm{M}}_n$ for their number.

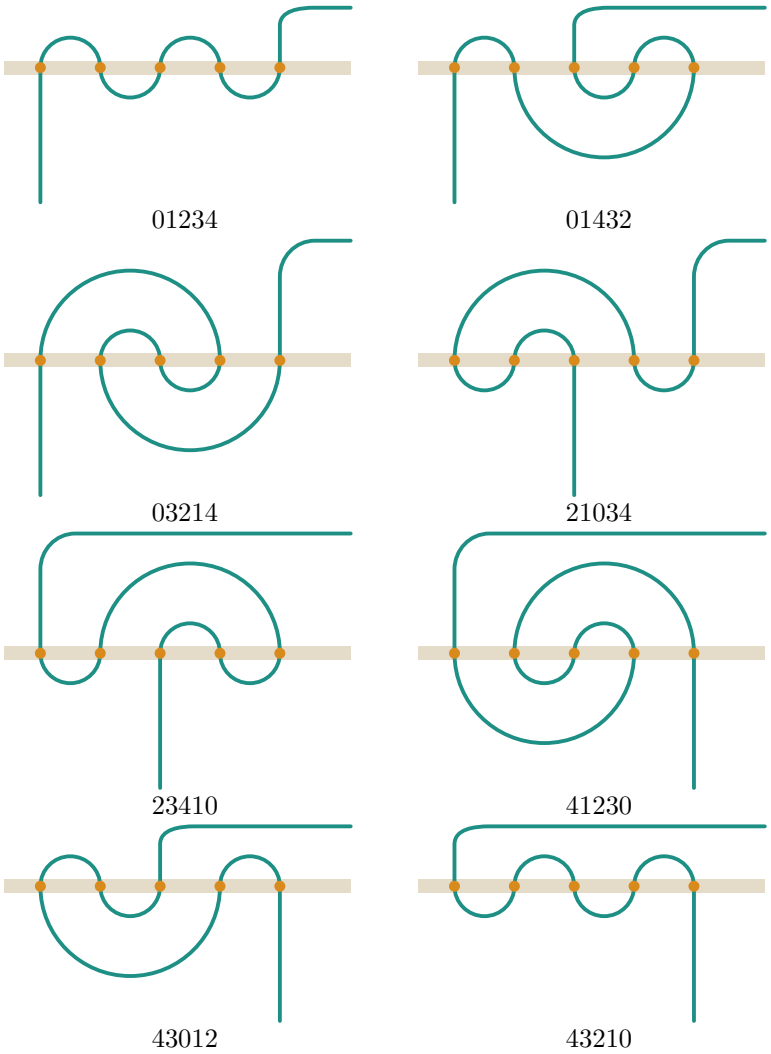


FIGURE 3. All eight open meanders with $n = 5$ crossings, each labelled with its crossing order (the bridges crossed, in river order, numbered from west to east). The river enters from the south; since n is odd, its last arc lies above the road and it leaves to the east above every other upper arc.

3. Structural results

Theorem 3.1. *For every n, $0 < \mathrm{M}_n \le n!$.*

Proof sketch. The upper bound is the cardinality of a subtype of the permutations. For the lower bound, the identity permutation is a meander: all its arcs except the first join neighbouring bridges, so nothing lies strictly between their ends, and the first arc, from $S = n + 1$ to bridge 0, encloses every other lower arc. □

Theorem 3.2. *For every $n \ge 1$, $\overline{\mathrm{M}}_n = \mathrm{M}_{2n-1}$.*

Proof sketch. Delete the westmost bridge of a closed meander and rotate the picture by 180°. The two arcs at the deleted bridge now run off to infinity: one in the lower half-plane, from which the river enters, and one to the east, where it leaves. The result is an open meander with $2n - 1$ crossings, and the construction can be reversed. In coordinates the map sends an open crossing order τ on $2n - 1$ bridges to the closed order 0, $2n - 1 - \tau(0)$, $2n - 1 - \tau(1), \ldots$, which in Lean is `decomposeFin.symm` $(0, \tau \cdot$ `revPerm`$)$. Reflection $x \mapsto 2n - 1 - x$ reverses the order on the bridges, so it preserves interleaving. The only subtlety is the south end $2n$, which truncated subtraction sends to the new bridge 0; it is an endpoint of a single lower arc, and every other lower arc has both endpoints among the bridges, so interleaving is unaffected. All of this reduces to `omega` once the values of the path are pinned down. □

Theorem 3.3. *For every n, $\overline{\mathrm{M}}_n \le C_n^2$, where $C_n = \frac{1}{n+1}\binom{2n}{n}$.*

Proof sketch. By Theorem 3.2 it suffices to inject open meanders with $2n + 1$ crossings into pairs of Dyck words of semilength $n + 1$; mathlib already proves that there are C_{n+1} such words. Treat the two ends of the river as extra points east of every bridge. Each half-plane then carries a non-crossing perfect matching of $2n + 2$ points, and the river is recovered by walking from the south end and taking, at each point, the partner on the side where the river currently is. A non-crossing matching is determined by its *pattern*, the word recording whether each point opens (U) or closes (D) its arc. To see this, let $s_i = \pm 1$ according to the pattern and let $B(x, z) = \sum_{x<i<z} s_i$. For an opener x with partner y, every arc starting strictly inside (x, y) also ends inside it, so $B(x, y) = 0$ and $B(x, z) \ge 0$ for $x < z \le y$. These two facts locate y from the pattern alone: if a second matching with the same pattern paired x with $y' > y$, then $B(x, y+1) = -1 < 0$, a contradiction. Closers are handled symmetrically. The pattern is a Dyck word because every closer in a prefix is matched to an opener in the same prefix. □

4. A verified search with parity pruning

The specification counts permutations, so evaluating it directly costs $n!$ steps. The first algorithm builds the crossing order one bridge at a time. It abandons a branch as soon as the newest arc crosses an earlier arc on the same side, or as soon as the following test fails.

Lemma 4.1 (Parity)**.** *Let q be a prefix of a meander, with $t = |q|$ arcs already drawn, and let $j < t$ be one of them, on side $s = j \bmod 2$, with ends $a < b$. Count the arc ends still to come on side s that lie strictly between a and b: every unused bridge in (a, b); the current point, if the next arc is on side s; and the east end, if the last arc is on side s. This number is even.*

Proof. Each later arc on side s has both ends inside (a, b) or neither, since it may not cross arc j. Each point listed is the end of exactly one later arc on side s, and no other point is. Formally, an induction along the remainder of the river shows that the running count of such ends inside (a, b) changes by 0 or 2 with each later arc on side s. □

n	M_n	crossing test	+ parity	+ reachability	completable prefixes
8	81	6,077	522	404	404
9	262	22,452	1,746	1,336	1,336
10	538	84,419	4,115	2,854	2,854
11	1,828	322,080	14,471	9,835	—

TABLE 1. Nodes visited by the depth-first search under successive pruning rules, and the number of prefixes that can actually be completed (a lower bound for any search that builds the river bridge by bridge).

The correctness theorem `openMeanderCount_eq_meanderCount` has two parts. First, the search equals the number of complete orderings, generated by the same recursion, that satisfy the specification; this holds because a pruned branch contains no meanders (for crossings, by monotonicity of `NoCross`; for parity, by Lemma 4.1). Second, permutations of $\mathrm{Fin}\, n$ correspond bijectively to members of that list of orderings.

Measured in a prototype, parity pruning reduces the number of search nodes at $n = 11$ from 322,080 to 14,471. A further rule, that every remaining bridge be reachable from the current point through the regions cut out by the drawn arcs, reduces this to 9,835. For $n \le 10$ the reduced search visits exactly the prefixes of actual meanders, so no further pruning rule of this kind can help (Table 1). In the compiled search the reachability test cost more than it saved, so we did not adopt it; the transfer matrix of the next section enforces connectivity by construction.

5. A verified transfer matrix

5.1. **The machine.** Scan the points of the road from west to east: the bridges, then E (one arc, on side n mod 2), then S (one arc, below). Each point *opens* or *closes* its arc on each of its sides. Arcs on one side never cross, so the arcs still open at a cut form a *stack* on each side, and a closing point always closes the top of its stack.

To the west of the cut the river is broken into pieces. Each piece joins two open arcs, or an open arc and the east end. The state records, for every open arc in stack order, a reference to where its piece leads:

```
inductive Ref where
  | E                          -- the east end
  | s (σ : Bool) (i : Nat)     -- position i in the stack of side σ

structure St where
  up : List Ref
  dn : List Ref
```

There are four transitions at a bridge. Opening both sides creates a new piece whose two ends are the two new arcs. Opening one side and closing the other continues the closed arc's piece along the new arc. Closing both sides joins two pieces, unless the two arcs being closed are the two ends of the same piece; that would close a loop, so the machine rejects it. At E the top arc on its side is closed and its piece now ends at E, or a new arc is opened towards S. A sequence is accepted when, after E, exactly one arc is open, below the road, leading to E, so that S can close it.

5.2. **Counting with merging and pruning.** The machine is run on all action sequences at once, layer by layer. Each layer is a list of states with multiplicities, and equal states

are merged by sorting and adding adjacent counts. Any merge procedure is admissible if it preserves weighted sums $\sum_{(s,c)} c \cdot g(s)$ for every weight function g; the counting theorem `tmCountWith_eq_countP` is proved for an arbitrary such procedure. We instantiate it twice: with a sort-based merge for compiled execution, and with a structurally recursive insertion merge that the Lean kernel can evaluate.

Each layer also drops states that cannot be finished. Every remaining point closes at most one arc on each of its sides, and the accepting state has no open arc above the road and one below, so a state with more open arcs than the remaining points can close has no accepted continuation. In a prototype this *viability* test reduces the total number of states over all layers at $n = 17$ from 512,350 to 3,135. Its soundness is an induction showing that a single step lowers each stack by at most one, and only on the sides the point has.

5.3. **Correctness.** The counting theorem reduces correctness of the transfer matrix to the following statement.

Theorem 5.1. *The action sequences accepted by the machine are in bijection with open meanders. Consequently* `tmCount m = openMeanderCount m` *for every* m.

The bijection is given explicitly. A meander τ is *encoded* by letting point y open its arc on side σ exactly when the other end of that arc lies east of y. An accepted sequence is *decoded* by running a decorated machine, described next, and reading off the river it has built. We prove that decoding an accepted sequence yields a meander whose encoding is the sequence, and that encoding a meander yields an accepted sequence that decodes back to the meander.

Decoding. The decorated machine stores, for every open arc, the point where it opened and its actual piece of river: the list of points from its origin, through scanned points, to the origin of its partner or to E. It also records the closed arcs. Forgetting the decorations gives back the transfer matrix exactly (lemma `step_abs`), so decorated runs and plain runs accept the same sequences. The heart of the proof is an invariant with twenty conjuncts, preserved by each of the five kinds of step (open–open, open–close for either side, close–close, and the two moves at E). It states, among other things, that partners point to each other; that a piece is duplicate-free, begins at its arc's origin, and is the reverse of its partner's piece; that consecutive points of a piece are joined by recorded arcs; that pieces of different components are disjoint and together cover every scanned point; that recorded arcs on one side do not interleave and do not straddle the origin of an open arc on that side; that each point is an arc end at most once per side; and that the recorded arcs agree with the actions read. At acceptance only one piece remains. Prefixing the south end gives a duplicate-free list of all $m + 2$ points in which consecutive points are joined by arcs. Because each point is an arc end at most once per side, the sides of consecutive arcs must alternate, and the non-interleaving of recorded arcs then yields the meander condition.

Encoding. Along the run of a meander's own sequence we maintain a second, simpler invariant: every recorded arc is an arc of the meander, and the stack on each side consists exactly of the scanned points whose arc on that side crosses the cut, in increasing order. The non-crossing property implies that a closing point's partner is the largest such point, hence the top of the stack. The only remaining danger is the loop check. Suppose a bridge closes both sides and the two arcs being closed are the ends of one piece. That piece is a duplicate-free chain of meander arcs from one path-neighbour of the bridge to the other, and every step changes the position along the river by exactly one. A discrete intermediate-value argument shows the chain passes through the position of the bridge itself, which is impossible because the bridge has not been scanned yet. Finally, after E the stacks are forced: nothing is open above, and below only the

Algorithm	largest n	time
search, crossing test only (compiled)	15	72 s
search with parity pruning (compiled)	15	5 s
	17	62 s
transfer matrix (compiled)	24	1.3 s
	28	11 s
	30	32 s
	32	93 s
search, kernel evaluation	9	30 s for $n = 9$ alone
transfer matrix, kernel evaluation	13	68 s for $n = 13$ alone

TABLE 2. Running times.

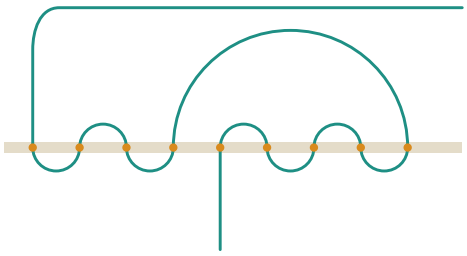


FIGURE 4. One of the 262 open meanders with $n = 9$ crossings; its crossing order is 456783210.

first bridge on the path is open, joined to S. Its piece is the whole river, so decoding returns the original meander.

5.4. **The proof in numbers.** The transfer-matrix development is about 3,650 lines of Lean. The project as a whole, including the specification, the structural theorems and the search, is about 4,960 lines and 171 theorems. All main theorems depend only on Lean's standard axioms (`propext`, `Classical.choice` and `Quot.sound`).

6. Certified values and performance

Table 2 compares the algorithms. Times are wall-clock on an Apple-silicon laptop for computing all values up to the given n. The specification itself is evaluable only for very small n.

The repository certifies values at two levels of trust. The statement

$$(\mathrm{M}_0, \ldots, \mathrm{M}_{13}) = (1, 1, 1, 2, 3, 8, 14, 42, 81, 262, 538, 1828, 3926, 13820)$$

is proved by evaluation in the Lean kernel, through Theorem 5.1 applied to the insertion-merge variant, and the corresponding closed values $(\overline{\mathrm{M}}_1, \ldots, \overline{\mathrm{M}}_7) = (1, 2, 8, 42, 262, 1828, 13820)$ follow by Theorem 3.2. Values up to $n = 24$ are proved with `native_decide`, which additionally trusts the Lean compiler. The compiled transfer matrix produces

$$\mathrm{M}_{30} = 1{,}503{,}962{,}954{,}930, \qquad \mathrm{M}_{32} = 15{,}012{,}865{,}733{,}351,$$

and all values for $n \le 32$ agree with the OEIS b-file. The executable is the same function that the correctness theorem is about, so these values carry the guarantee of Theorem 5.1 up to the correctness of compilation.

6.1. **All implementations.** The repository contains the algorithms in five languages. Only the Lean programs are verified; the others run the same machine and are checked against the certified values and the OEIS.

- *Lean.* The specification (evaluable by brute force for $n \leq 6$), the search `meanderCount` with its array implementation substituted by `csimp`, and the transfer matrix in two variants: `tmCount`, compiled into the executable `meanders`, and `tmCountK`, evaluated by the kernel.
- *Rust* (`rust/`). The transfer matrix with a packed state. Read along the cut from top to bottom, the open arcs pair up like brackets, because the pieces of river west of the cut never cross. So a state is a bracket word with at most one marker for the arc that leads to the east end, plus the number of arcs above the road, and fits in 64 bits for $n \leq 52$. Each layer is split into 4096 hash shards, each an open-addressing table behind its own lock; worker threads claim source shards, batch successors per target shard, and free each source shard once read, so memory holds about one layer. Tables grow on demand, or, with an option, are allocated once at a size predicted from the previous layers' growth. Counts are kept modulo 2^{64} and, from $n = 44$, also modulo $2^{61} - 1$, and recovered by the Chinese remainder theorem.
- *OxCaml* (`oxcaml/`). The same algorithm in OxCaml, Jane Street's extension of OCaml. Workers run under OxCaml's `Parallel` scheduler, shards are capsules guarded by mutexes, and the compiler's mode system rules out data races. The state is an immediate 63-bit integer, the hot loop allocates nothing on the heap, and tables are flat arrays of unboxed values that the garbage collector never scans.
- *Python and OCaml.* Companion notebooks teach each language by building the specification, the parity search and the transfer matrix from scratch, with the state as a string or a list of constructors and layers in hash tables, run on all cores with process pools (Python) and domains (OCaml 5). A third notebook teaches Lean with the library's own declarations.

Table 3 times the parallel programs on a workstation with an 18-core Intel Xeon W-2191B (36 threads) and 128 GB of memory, one value per run on all threads; on the same machine the verified `meanders` computes all values up to $n = 32$ in 160 s. Every count equals the OEIS term; in particular

$$\mathrm{M}_{49} = 8{,}499{,}066{,}628{,}515{,}413{,}229{,}282, \qquad \mathrm{M}_{50} = 22{,}206{,}891{,}674{,}746{,}169{,}557{,}410.$$

Memory is the limit: the largest layer for $n = 50$ holds 2.25×10^9 states, and $n = 50$ is the largest value that fits in 128 GB, at about 35 bytes per state. Table 4 shows that the programs scale to the physical cores and no further, as expected for work dominated by random memory access.

The OxCaml program is faster than the Rust program in its default mode, and the difference is one design choice. By default Rust grows each shard's table by half when it is 80% full, so every state is moved several times as its table grows; OxCaml predicts each layer's size from the growth of the previous layers and allocates every table once. With the same prediction (Rust's pre-sizing option), Rust is 20–28% faster at every thread count, including one, so the saving is the rehashing work itself rather than time spent waiting on locks. Pre-sized Rust is then the fastest program for $n \geq 44$ and on a single thread; OxCaml keeps an edge for small n on many threads, plausibly because it keeps one pool of workers and their buffers for the whole run, while Rust starts its threads and allocates their batch buffers for every point. Pre-sizing costs memory, since each table is allocated at its final size while the previous layer is still draining (17.0 against 12.6 GiB at $n = 46$); OxCaml needs more still (20.5 GiB), because

n	M_n	largest layer	Rust	pre-sized	OxCaml
40	1.66×10^{17}	2.2×10^{7}	3.6 s	2.9 s	2.2 s
42	1.73×10^{18}	5.7×10^{7}	6.1 s	5.3 s	5.3 s
44	1.83×10^{19}	1.4×10^{8}	30 s	22 s	25 s
46	1.94×10^{20}	3.5×10^{8}	80 s	57 s	65 s
49	8.50×10^{21}	1.4×10^{9}	516 s		
50	2.22×10^{22}	2.3×10^{9}	1052 s		

TABLE 3. The parallel programs on 36 threads of an 18-core Xeon with 128 GB, interleaved runs (Rust with tables grown on demand, Rust with pre-sized tables, OxCaml). From $n = 44$ a value takes two sweeps. Peak memory at $n = 46$: 12.6, 17.0 and 20.5 GiB; at $n = 50$: 74 GiB (Rust).

threads	1	4	9	18	36
Rust	21.3 s	6.1 s	3.5 s	3.0 s	3.3 s
Rust, pre-sized	17.0 s	4.8 s	2.8 s	2.4 s	2.9 s
OxCaml	20.4 s	5.6 s	2.8 s	2.2 s	2.3 s

TABLE 4. Scaling with threads for $n = 40$ on the same machine.

its freed tables are returned only after a major garbage collection. Rust grows on demand by default, so that the largest n fits in memory. The teaching implementations are far slower: interpreted Python computes all values up to $n = 30$ in about twelve seconds on the laptop, and the OCaml notebook's transfer matrix, run as bytecode on ten domains, takes about half a second for $n = 28$.

6.2. **Meet in the middle and new values.** The transfer matrix keeps one layer at a time, and memory stops it at $n = 50$. Meeting in the middle trades that memory for disk and reuses work across n.

- *Forward layers.* Let F_k be the layer of states after k bridges from the west, with no pruning. It does not depend on n.
- *Backward layers.* Let $G_r^p(s)$ be the number of ways to finish a river from state s in r further bridges, with the east end on side p of the road.
- *The decomposition.* For every split k,

$$M_{k+r} = \sum_s F_k(s)\, G_r^{(k+r) \bmod 2}(s).$$

  The Lean theorem `meet` (`Arnold/TM/Middle.lean`) proves this for the verified machine, through `cntFrom_eq_finish`, which identifies the backward counts with the forward machine run from s. The kernel checks the identity numerically for $n \le 10$ (`openMeanderCount_values_meet`).

The programs build the backward layers by inverting the bridge steps, which is not verified. They keep a state in G_r only if its *depth*, the fewest bridges that build it from the west, is at most $B - r$ for the horizon B. Deeper states cannot meet any forward layer within the horizon, so the pruning is exact. Every state of these layers is a bracket word without the marker for the east end, so both programs pack it into one machine word. OxCaml also drops the first and last brackets, which are always "(" and ")", so that 58 brackets fit in its 63-bit integers.

	Rust, first	Rust, ported	OxCaml
store, $B = 52$	2953 s	1055 s	1272 s
in memory, all $n \le 46$	72 s	40 s	44 s
in memory, all $n \le 48$		92 s	100 s

TABLE 5. Meet in the middle on 32 threads of the workstation, before and after porting OxCaml's sort, buffer reuse and table handling to Rust. Kernel CPU time for the store: 24,581 s (Rust, first), 1,934 s (Rust, ported), 2,135 s (OxCaml).

Both programs keep every layer on disk as a few large files of sorted, delta-encoded records. A layer that does not fit in memory spills sorted runs and merges them. Raising the horizon from B to B' adds one segment to each backward layer, holding the states that B' newly admits, and recomputes nothing. Each M_n is computed with $k = \lceil n/2 \rceil$ and checked with $k-1$, which uses different forward and backward layers. Counts are exact: 128-bit integers in Rust, and two 62-bit limbs in OxCaml.

On the workstation of Table 3:

- The Rust store reached $B = 58$ by stepwise extension from $B = 48$.
- The OxCaml store was built directly at $B = 58$ in 8.0 hours. Its largest backward layer holds 3.6×10^{10} states, and F_{29} holds 2.6×10^{9}.
- The stores take 1.3 TB each.

Every value $n \le 58$ agrees between its two splits and between the two programs, and the values for $n \le 55$ agree with the OEIS b-file. The new values are

$$\mathrm{M}_{56} = 28{,}235{,}899{,}288{,}344{,}793{,}178{,}333{,}732,$$
$$\mathrm{M}_{57} = 116{,}936{,}079{,}373{,}841{,}873{,}422{,}508{,}566,$$
$$\mathrm{M}_{58} = 308{,}496{,}356{,}015{,}441{,}090{,}010{,}351{,}094.$$

In memory, the same algorithm computes every value up to $n = 48$ on the workstation in 92 s (Rust) and 100 s (OxCaml). Table 3 gives the transfer matrix 80 s for $n = 46$ alone. The forward layers serve every n at once.

The two implementations raced again on this algorithm (Table 5).

- *OxCaml first.* OxCaml's store was at first 2.3 times faster at $B = 52$. Rust spent more CPU time in the operating system than in its own code, for three reasons:
  - a comparison sort of every spilled table;
  - a full table emptied while its lock was held;
  - fresh large buffers for every spill, sort and read, each a new memory mapping that is faulted in again.
- *Why OxCaml had the better design.* OxCaml, written second and profiled on purpose, used a radix sort and per-worker buffers from the start.
- *Rust with the same design.* With those choices ported, together with pre-sized tables and parallel dot products in memory, Rust is 8–17% faster than OxCaml and uses slightly less memory.

This repeats the conclusion for the transfer matrix: with the same design the two languages are close, and the lead goes to whichever program had the more careful second look.

7. Discussion

7.1. **What the proofs say about the algorithms.** The two pruning rules are the computational shadows of the Jordan curve theorem. Parity (Lemma 4.1) says that arcs pair up inside every region; viability says that a point can close at most one arc per side. Neither rule needs any topology beyond the interleaving of chords, which is why both proofs reduce to linear arithmetic.

The transfer-matrix proof is longer, mostly because the machine's state records connectivity abstractly, by reference, while correctness needs to know what is connected to what. The decorated machine bridges the gap: it carries explicit pieces of river, so the connectivity argument becomes reasoning about lists. Making the abstract machine literally the decorated one with decorations forgotten (`step_abs`) means no separate refinement proof is needed.

7.2. **Meandric permutations and the conjectured scaling limit.** The specification is a statement about permutations, and the same permutations appear in recent probability. Borga, Gwynne and Sun [3] attach to a closed meander its *meandric permutation*: number the crossings from west to east, and again in the order in which the river meets them; the permutation relates the two orders. Up to inversion and the choice of where and in which direction the river is started, this is the permutation σ of `IsClosedMeander`, and Rosenstiehl [13] had studied the same objects as planar permutations. The formalization therefore counts meandric permutations exactly, and Theorem 3.2 is a bijection between meandric permutations of size $2n$ and open meanders with $2n-1$ crossings.

Borga, Gwynne and Sun conjecture that a uniform random meander, viewed as a planar map with two Hamiltonian paths (the road and the river), converges to a Liouville quantum gravity surface with matter central charge -4, that is $\gamma = \sqrt{(17-\sqrt{145})/3} \approx 1.29$, decorated by two independent space-filling SLE_8 curves, and that the plots of uniform meandric permutations converge to the corresponding *meandric permuton*. They prove, among other things, that the longest increasing subsequence of permutations sampled from the permuton is sublinear. The conjecture is the probabilistic counterpart of the asymptotic formula of Di Francesco, Golinelli and Guitter [5],

$$\overline{\mathrm{M}}_n \sim C\, A^n\, n^{-\alpha}, \qquad \alpha = \frac{29+\sqrt{145}}{12} \approx 3.4201,$$

for the number $\overline{\mathrm{M}}_n$ of closed meanders of order n, with $A \approx 12.2629$ estimated by Jensen and Guttmann [6, 7].

Exact counts test the exponent directly. With A fixed at that estimate,

$$\alpha_n = -\frac{\log\bigl(\overline{\mathrm{M}}_{n+1}/(A\overline{\mathrm{M}}_n)\bigr)}{\log(1+1/n)}$$

estimates α, with an error of order $1/n$; extrapolating linearly in $1/n$ from α_{n-2} and α_n removes most of it. Table 6 uses $\overline{\mathrm{M}}_n = \mathrm{M}_{2n-1}$ (Theorem 3.2) with open counts up to 57 crossings, computed by the programs of Sections 6.1 and 6.2; the rows for $n = 26$ and 28 use the new value $\overline{\mathrm{M}}_{29} = \mathrm{M}_{57}$. The extrapolated values increase steadily towards 3.4201. This is numerical evidence for the conjectured exponent, not a proof, and it assumes the estimate of A.

7.3. **Limitations and further work.** Our verified transfer matrix is far from the state of the art. Jensen's algorithm [6] encodes states more compactly and enumerates well beyond $n = 32$. The unverified Rust and OxCaml programs of Section 6.1 pack each state into one machine word and reach $n = 50$, and their meet-in-the-middle stores reach $n = 58$. Verifying

n	$\overline{\mathrm{M}}_{n+1}/\overline{\mathrm{M}}_n$	α_n	extrapolated
12	9.4415	3.2665	3.4053
14	9.7747	3.2870	3.4100
16	10.0377	3.3027	3.4129
18	10.2506	3.3152	3.4149
20	10.4263	3.3253	3.4163
22	10.5739	3.3337	3.4172
24	10.6996	3.3407	3.4180
26	10.8078	3.3467	3.4185
28	10.9021	3.3518	3.4189
		conjecture [5]	3.4201

TABLE 6. Estimates of the meander exponent α from exact counts of closed meanders, with the growth constant fixed at $A = 12.2629$.

such an encoding is a natural next step, as is verifying the inverse bridge steps and the depth pruning on top of the proved decomposition; the generic counting theorem already accepts any merge procedure that preserves weighted sums. The stores could go beyond $n = 58$ with a wider state key and more disk. A second direction is to formalize the rigorous growth-rate bounds of Albert and Paterson [1], which would make our Catalan bound one instance of a more general framework. Finally, the question that motivated the work, a closed formula for the meander numbers, remains open; the conjectured critical exponent $\alpha = (29 + \sqrt{145})/12$ suggests that if a formula exists it is not of an elementary kind.

Acknowledgements. The formalization and this paper were developed with the assistance of Claude Code (Anthropic).

REFERENCES

[1] M. H. Albert and M. S. Paterson, *Bounds for the growth rate of meander numbers*, J. Combin. Theory Ser. A **112** (2005), 250–262.
[2] V. I. Arnold, *The branched covering $\mathbb{CP}^2 \to S^4$, hyperbolicity and projective topology*, Siberian Math. J. **29** (1988), 717–726.
[3] J. Borga, E. Gwynne and X. Sun, *Permutons, meanders, and SLE-decorated Liouville quantum gravity*, J. Eur. Math. Soc. **28** (2026), 4893–4949; arXiv:2207.02319.
[4] P. Di Francesco, O. Golinelli and E. Guitter, *Meander, folding, and arch statistics*, Math. Comput. Modelling **26** (1997), 97–147.
[5] P. Di Francesco, O. Golinelli and E. Guitter, *Meanders: exact asymptotics*, Nuclear Phys. B **570** (2000), 699–712.
[6] I. Jensen, *A transfer matrix approach to the enumeration of plane meanders*, J. Phys. A **33** (2000), 5953–5963.
[7] I. Jensen and A. J. Guttmann, *Critical exponents of plane meanders*, J. Phys. A **33** (2000), L187–L192.
[8] S. K. Lando and A. K. Zvonkin, *Plane and projective meanders*, Theoret. Comput. Sci. **117** (1993), 227–241.
[9] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, in Automated Deduction – CADE 28, Lecture Notes in Comput. Sci. **12699**, Springer, 2021, 625–635.
[10] The mathlib Community, *The Lean mathematical library*, in Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), 367–381.
[11] OEIS Foundation, *Sequence A005316: Meandric numbers*, The On-Line Encyclopedia of Integer Sequences, `https://oeis.org/A005316`.
[12] OEIS Foundation, *Sequence A005315: Closed meandric numbers*, The On-Line Encyclopedia of Integer Sequences, `https://oeis.org/A005315`.

[13] P. Rosenstiehl, *Planar permutations defined by two intersecting Jordan curves*, in Graph Theory and Combinatorics (Cambridge, 1983), Academic Press, London, 1984, 259–271.
[14] J. Touchard, *Contributions à l'étude du problème des timbres-poste*, Canad. J. Math. **2** (1950), 385–398.

QueryGraph